🌾 Digital Sarathi v1.0 – Finalized System Plan

🎯 Core Objective

A multi-agent, personalized agricultural decision system that:

- Gives ready recommendations proactively
- Adapts dynamically to constraints (not rule-based)
- Provides practical, real-world feasible advice
- Acts as a consultant, not just a predictor

🧠 1. Intelligence Model (Final)

🔥 Dynamic Decision Engine (Finalized)

This system does NOT follow fixed hierarchy.

Instead:

- Every agent contributes
- Constraints are evaluated contextually
- Output = Best possible decision under current + potential conditions

---

🧩 Decision Logic

Step 1: Generate feasible crops
Step 2: Score across all agents
Step 3: Detect conflicts
Step 4: Evaluate:
    - Can constraint be resolved?
    - Is it worth resolving?
Step 5: Produce:
    - Direct recommendation
    - Conditional recommendation
    - Alternative paths

🤖 2. Multi-Agent Architecture (LangGraph)

🧠 **Agents (Final Set)**

1. Soil Agent

- **NPK, pH, soil type**
- **Outputs: Suitability score**

---

2. Crop Knowledge Agent

- **Indian crop dataset (≤ 100 crops)**
- **Filters viable crops**

---

3. Water & Climate Agent

- **Weather API + farmer water data**
- **Outputs:**
  - **Feasibility**
  - **Risk level**

---

4. Market Intelligence Agent

- **Price signals via eNAM**
- **Basic trend awareness**

---

5. Policy & Subsidy Agent (RAG)

- **Govt schemes like PMFBY**
- **Irrigation subsidies**

---

6. Farmer Profile Agent (Memory Core)

Stores:

- **Land (acres)**
- **Water sources**
- **Budget**

- Risk appetite
- Equipment
- Crop history
- Income dependency

---

7. Sustainability Agent

- Crop rotation
- Soil preservation

---

8. Pest Advisory Agent

- Adds risk alerts (non-blocking)

🔹 Orchestrator (LangGraph Flow)

Input
→ Fetch Farmer Profile
→ Generate Candidate Crops
→ Parallel Agent Scoring
→ Conflict Resolution Engine
→ Recommendation Generator

⚖️ 3. Conflict Resolution Engine (Final)

💡 Key Behavior:

- No agent dominates
- No blind rejection
- Always tries adaptive reasoning

🧠 Example Outcome:

"Cotton is highly suitable for your soil.
However, current water levels make it risky.

If you proceed without irrigation → high failure risk

If you install drip irrigation (eligible for subsidy) → Cotton becomes viable and profitable

Alternatively, Sorghum offers stable returns under current conditions."

✓ Shows conflict
✓ Resolves it
✓ Gives options

👤 4. Farmer Profile System (Full Personalization)

Stored Attributes:

- **Land size (acres)**
- **Water type (rain-fed / borewell / canal)**
- **Budget range**
- **Risk level (Low / Medium / High)**
- **Equipment availability**
- **Past crops (2–3 seasons)**
- **Income dependency**

Behavior Impact:

- **Low budget → avoids high-cost crops**
- **High risk → suggests aggressive options**
- **Legacy crops → preferred unless strong reason to change**

📊 5. Recommendation System (Final Output Design)

🌱 System ALWAYS gives:

Option A – Safe

- **Low investment**
- **Stable yield**

Option B – High Reward

- **Higher investment**
- **Higher return potential**

Option C – Soil Health

- **Improves long-term fertility**

🔁 **Default Mode:**

- **System proactively generates recommendation (based on profile)**

🔀 **On Demand:**

- **Farmer can ask:**
  - **"Split land planning"**
  - **"Alternative crops"**
  - **"More profit options"**

💬 **6. Consultant Mode (Text-Based Interaction)**

Behavior:

- **AI asks context-aware questions**

Example:

AI:

"Your soil is suitable for high-yield crops. How reliable is your water supply this season?"

User:

"Low"

AI:

"Adjusting recommendations to reduce water dependency while maintaining returns."

✓ Feels like expert
✓ Not form-based
✓ Adaptive flow

📚 7. RAG Layer (Minimal but Effective)

Ingest:

- Govt scheme docs
- Crop SOP guides
- Pest info

Usage:

- Subsidy suggestions
- Basic how-to guidance
- Grounded outputs

🛡️ 8. Pest/Disease Advisory (Final)

✓ Included as non-blocking layer

Example:

"Groundnut is suitable. Note: Leaf spot disease is common in your region during October. Preventive measures recommended."

🖥️ 9. UI/UX – Streamlit (Phase 1)

Screens:

1. Profile Setup

- Farmer inputs all details

---

2. Dashboard

- **Shows:**
    - **Recommended crops**
    - **Option A/B/C**

---

3. Consultant Chat

- **Interactive advisory**

---

4. History

- **Past recommendations**

🎤 **10. Voice System (Designed, Not Implemented Yet)**

Key Concept:

- **Voice model depends on farmer's setup**

Configurable in Profile:

- **Language**
- **Model type (lightweight / advanced)**

Future Compatible With:

- **Whisper / Vosk (input)**
- **Coqui / Google TTS (output)**

✓ Flexible
✓ Device-aware
✓ User-controlled

⚙️ **11. Tech Stack (Final)**

Backend:

- Python
- LangChain
- LangGraph

RAG:

- FAISS / Chroma

APIs:

- Weather → OpenWeather
- Market → eNAM (minimal usage)

Frontend:

- Streamlit

🧪 12. MVP Scope (Strict)

Include:

✓ Multi-agent reasoning
✓ Dynamic conflict resolution
✓ Full farmer profile
✓ Minimal RAG
✓ Consultant interaction
✓ 3-option output
✓ Indian crop dataset (≤100)

Avoid:

❌ Over-complex UI
❌ Heavy simulations
❌ Too many APIs

🚀 13. Future Enhancements

Phase 2:

- Voice assistant (multilingual)

Phase 3:

- What-if simulation

Phase 4:

- Market prediction

Phase 5:

- Community intelligence (avoid crop glut)

Phase 6:

- Crop lifecycle tracking + reminders

🧩 14. Final Identity

This system is:

✓ A Decision Support System
✓ A Digital Agricultural Consultant
✓ A Personalized Planning Engine

🔥 Final Statement

You are not building:

"An AI that predicts crops"

You are building:

A system that thinks like an agricultural expert, plans like a strategist, and adapts like a human advisor